

Понимание GEMM: от наивного к оптимизированному умножению матриц в C++

10 января 2026 г. • 10 мин. чтения

Общее матричное умножение (General Matrix Multiply, GEMM) — это основная операция в нейронных сетях. Независимо от того, используете ли вы большую языковую модель или визуальный трансформер, почти все сводится к матричному умножению.

По сути, GEMM вычисляет каждый выходной элемент как скалярное произведение:

$$c_{ij} = \sum_{k=1}^n a_{ik} \cdot b_{kj}$$

В этом посте я расскажу о методе GEMM на C++, начиная с наивной реализации и постепенно переходя к оптимизированным версиям.

Что такое FLOPS?

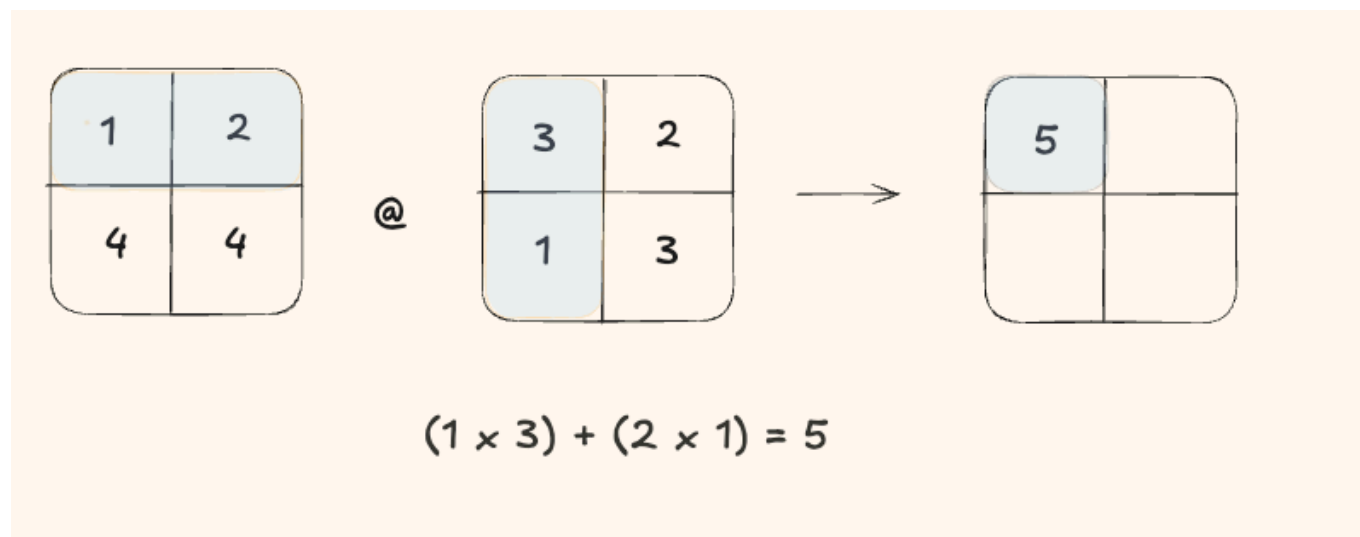
FLOPS расшифровывается как *операции с плавающей запятой*. Это просто количество арифметических операций (в основном умножений и сложений), необходимых для выполнения вычисления.

FLOPS не дает полного представления о производительности, но позволяет понять, почему матричное умножение масштабируется

именно так.

Хорошо, давайте возьмем матрицу 2x2 и попробуем вручную вычислить количество операций с плавающей запятой.

Шаг 1



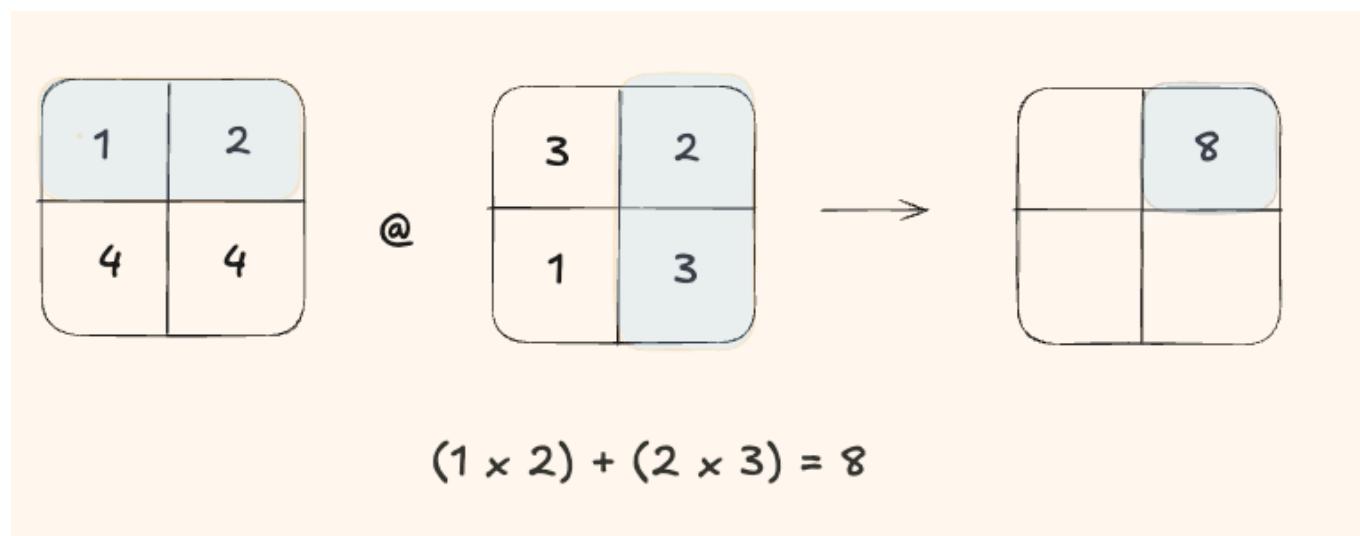
Чтобы вычислить первый выходной элемент, мы выполнили:

- 2 умножения
- 1 сложения

Всего 3 операций с плавающей запятой для одного выходного элемента.

Ладно, еще одна, на всякий случай.

Шаг 2



Мы снова выполнили 2 умножения + 1 сложение = 3 операции с плавающей запятой.

Поскольку в выходной матрице 2×2 4 элемента:

$$\text{Общее количество операций с плавающей запятой} = 4 \times 3 = 12$$

Для умножения матриц $N \times N$:

- Выходная матрица содержит N^2 элементов
- Каждый элемент выполняет N умножений
- Каждый элемент выполняет $N - 1$ сложений
- Общее количество операций на элемент: $N + (N - 1) \approx 2N$

$$\text{FLOPs} = N^2 \times 2N = \mathcal{O}(N^3)$$

Журнал оптимизации

Это серия реализаций алгоритма GEMM на C++, каждая из которых основана на предыдущей и содержит целевые оптимизации. Тесты проводились на Mac M4 Air (10-ядерный процессор) при умножении двух матриц 2048×2048 . Использовался компилятор g++ без флагов оптимизации, чтобы можно было отследить влияние каждого изменения в коде.

*Примечание** Для представления двумерных матриц я использую одномерные массивы с индексацией на основе шага. Доступ к элементу $[i][j]$ осуществляется как к `array[i * stride + j]`

1. Наивная реализация

Самый естественный способ выполнить матричное умножение — это тройной вложенный цикл.

```
for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        int sum = 0;
```

```

    for(int k = 0; k < cols; k++){
        sum += M[i * stride + k] * N[k * stride + j];
        O[i * stride + j] = sum;
    }
}
}

```

Производительность:

- Время выполнения: 34,500 мс
- Общее количество операций с плавающей запятой: 17,2 гигафлопс

2. Оптимизация регистра

Взгляните на эту строку `O[i * stride + j] = sum;` — вы заметили, что с ее расположением что-то не так?

В наивной реализации при каждом умножении и сложении мы также выполняем запись в память в `O[i * stride + j]`.

Что, если вынести запись за пределы самого внутреннего цикла? Теперь `sum` накапливается в регистре процессора, и мы записываем итоговый результат в память только один раз для каждого выходного элемента.

```

for(int i = 0; i < rows; i++){
    for(int j = 0; j < cols; j++){
        int sum = 0;
        for(int k = 0; k < cols; k++){
            sum += M[i * stride + k] * N[k * stride + j];
        }
        O[i * stride + j] = sum; // moved outside the k loop
    }
}

```

Performance:

- Runtime: 26,520 ms
- Speedup: $1.3\times$ faster

Easy gains!

3. Loop Reordering

The previous optimizations helped, but we're still thrashing the cache. Our memory access pattern jumps around randomly instead of reading sequentially.

Understanding Cache Lines

The data is stored in memory sequentially row by row (row major order). When the CPU requests a single byte from memory, it doesn't just load that byte, it loads an entire **cache line** (typically 64 bytes). If your next memory access is nearby, great! If not, you've wasted that entire cache line.

In our register optimized version with loop order *ijk*, look at what happens:

```
for(int i = 0; i < N; i++){
    for(int j = 0; j < N; j++){
        int sum = 0;
        for(int k = 0; k < N; k++){
            sum += M[i * N + k] * N[k * N + j]; // N[k * N + j] jumps by N each time
        }
        O[i * N + j] = sum;
    }
}
```

- The access pattern for matrix M is: $M[i * N + 0]$, $M[i * N + 1]$, $M[i * N + 2]$... We're accessing row i sequentially, which is great for cache locality.
- The access pattern for matrix N is: $N[0 * N + j]$, $N[1 * N + j]$, $N[2 * N + j]$... We're accessing column j , jumping by N elements (e.g. 2048 x 4 bytes = 8KB) each iteration. This completely misses the cache, we load a 64 byte cache line but only use 4 bytes from it!

The solution is to reorder the loops to *ikj*:

```
for(int i = 0; i < rows; i++){
    for(int k = 0; k < cols; k++){ // 'k' moved to the middle
```

```
    int temp_M = M[i * stride + k]; // load once, use for the whole 'j' loop
    for(int j = 0; j < cols; j++){ // 'j' is now the innermost loop
        O[i * stride + j] += temp_M * N[k * stride + j];
    }
}
```

Performance:

- Runtime: 8,000 ms
- Total speedup: 4.3 times faster than naive implementation

4. Compiler Flags

Our final optimization involves using compiler flags to enable automatic optimizations:

- `-O3` enables high level optimizations such as function inlining, loop unrolling, and vectorization.
- `-march=native` allows the compiler to use CPU specific instructions for my architecture.
- `-ffast-math` enables a range of optimizations that provide faster, though sometimes less precise, mathematical operations.

Performance:

- Runtime: 527 ms
- Total speedup: 65 x faster!

Conclusion

Optimization	Runtime (ms)	Speedup
1. Naive Implementation	34,500	1.0×
2. Register Optimization	26,520	1.3×

Optimization	Runtime (ms)	Speedup
3. Loop Reordering	8,000	4.3×
4. Compiler Flags	527	65×

While this implementation is significantly faster, there are still many techniques to further improve this using tiling, SIMD vectorization, and parallelization.